



Week4_예습과제_고은비

Chapter4. 분류

- 5 GBM (Gradient Boosting Machine)
 - GBM 하이퍼 파라미터
- 6 XGBoost(eXtra Gradient Boost)
 - 파이썬 래퍼 XGBoost 하이퍼 파라미터
 - 사이킷런 래퍼 XGBoost의 개요 및 적용
- 7 LightGBM
 - LightGBM 하이퍼 파라미터
 - 하이퍼 파라미터 튜닝 방안
 - 파이썬 래퍼 LightGBM과 사이킷런 XGBoost, LightGBM 파라미터 비교
- 8 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝
 - 베이지안 최적화 개요
 - HyperOpt 사용하기
- 10 분류 실습 - 언더 샘플링과 오버 샘플링의 이해
- 11 스택킹 앙상블
 - CV 세트 기반의 스택킹

Chapter4. 분류

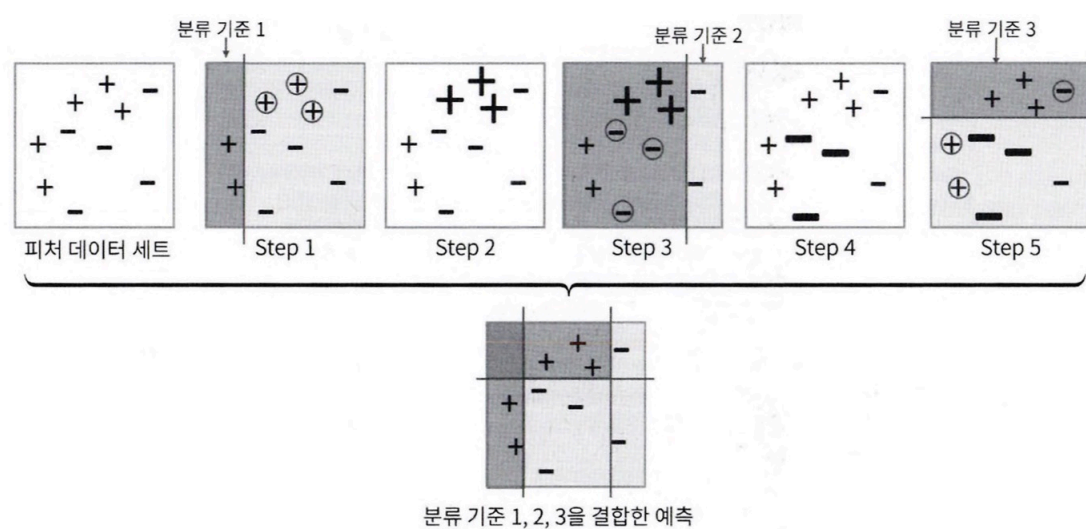
5 GBM (Gradient Boosting Machine)



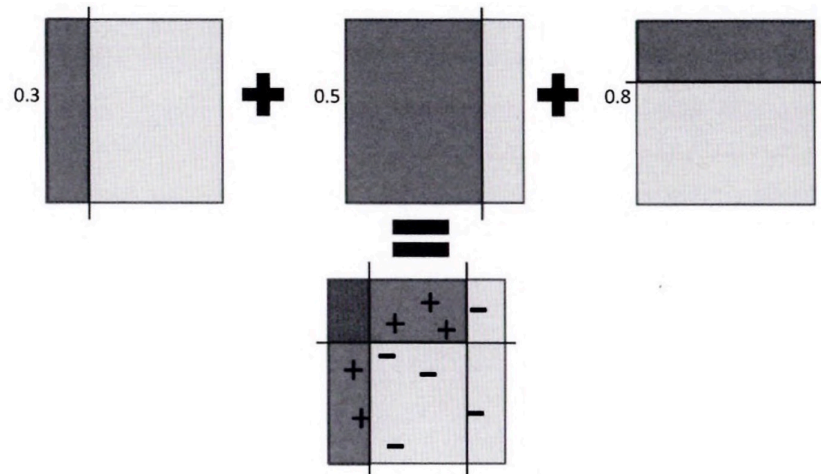
GBM이란?

- 부스팅 알고리즘 : 여러개의 약한 학습기 (weak learner)를 순차적으로 학습 - 예측하면서 잘못 예측한 데이터에 가중치 부여를 통해 오류를 개선해 나가면서 학습하는 방식
- ⇒ Ada Boost와 함께 대표적인 부스팅의 구현

AdaBoost (Adaptive boosting)



- Step1 첫 번째 약한 학습기가 분류기준 1으로 +와 -를 구분한 것 (동그라미로 표신도니 데이터가 오분류 데이터)
 - Step2 오류 데이터에 대한 가중치 값 부여 → 다음 학습기가 더 잘 분류하도록 크기가 커짐
 - 반복
- ⇒ 마지막으로 모든 분류 기준을 모두 결합 (개별 학습기보다 정확도가 높아짐)



- 개별 학습기를 결합 시 - 각각 가중치를 부여해 결합

GBM

에이다 부스트와 유사하나, 가중치 업데이트를 **경사 하강법(Gradient Descent)**을 이용함



경사 하강법 ?

오류 값은 실제 값에서 예측값을 뺀 것

⇒ 실제 결과 값을 y , 피처를 $x_1, x_2, \dots, x_n$, 그리고 예측함수를 $F(x)$ 라고 할 때 오류 식은

$$h(x) = y - F(x)$$

이 오류식을 최소화하는 방향성을 가지고 반복적으로 가중치 값을 업데이트하는 방법

- 분류는 물론, 회귀도가능함
- 사이킷런에서는 `GradientBoostingClassifier` 클래스를 제공
- 랜덤 포레스트보다 뛰어난 성능을 보이지만, 수행 시간 문제가 존재
 - ⇒ 약한 학습기의 순차적인 예측 오류 보정을 통해 학습을 수행하므로, 멀티 CPU코어 시스템을 사용하더라도 병렬처리가 지원되지 않음
 - ↔ 랜덤 포레스트는 상대적으로 빠른 수행 시간을 보장



GBM 하이퍼 파라미터

하이퍼 파라미터	설명
<code>loss</code>	경사 하강법에서 사용할 비용 함수를 지정 → 특별한 이유가 없으면, 기본값 'deviance'를 그대로 적용
<code>learning_rate</code>	GBM이 학습을 진행할 때마다 적용하는 학습률 - Weak learner가 순차적으로 오류 값을 얼마나 보정해 나갈지 ▪ 0 ~ 1 사이의 값을 지정 가능, 기본값은 0.1 ▪ 너무 작은 값을 적용하면, 업데이트되는 값이 작아져서 최소 오류 값을 잘 찾아 성능이 높아질 수 있지만, 수행시간이 오래걸리고 최소 오류 값을 찾지 못할 수 도 있음 ▪ 반대로 큰 값을 적용하면, 최소 오류 값을 지나쳐 버려서 성능이 떨어질 수 있음 하지만, 빠르죠? ➡ <code>n_estimators</code> 와 상호 보완적으로 조합해 사용한다. 학습률을 작게하고 <code>n_estimators</code> 를 크게 하면 예측 성능이 조금 씩 좋아질 수 있다. 근데 수행 시간이 오래걸리며, 현격히 좋아지진 않음
<code>n_estimators</code>	weak learner의 개수 (기본 100개) ⇒ 많을수록 예측 수준이 일정 수준 좋아지지만, 수행시간도 늘어남
<code>subsample</code>	weak learner가 학습에 사용하는 데이터 샘플링의 비율 (기본 1) ⇒ 과적합이 우려되는 경우, 1보다 작은 값으로 설정하기

6 XGBoost(eXtra Gradient Boost)



XGBoost?

트리 기반의 앙상블 학습에서 가장 각광받고 있는 알고리즘 중 하나로, 분류에 있어 다른 머신러닝보다 뛰어난 예측 성능을 나타낸다.

GBM에 기반하고 있지만, GBM의 단점인 느린 수행 및 과적합 규제의 부재 등의 문제를 해결해서 매우 각광을 받고 있음. 특히 병렬 GPU 환경에서 병렬 학습이 가능!

- 핵심 라이브러리는 C/C++로 작성 → 파이썬 패키지 "xgboost" 제공 (사이킷런과 연동X)
⇒ 사이킷런과의 호환을 위해 Wrapper class 제공 (XGBClassifier, XGBRegressor)
- 사이킷런 래퍼 모듈은 사이킷런의 다른 Estimator와 사용법이 동일 / 파이썬 네이티브는 고유의 API와 하이퍼 파라미터를 사용

XGBoost 주요 장점

- **뛰어난 예측 성능**
일반적으로 분류와 회귀 영역에서 뛰어난 예측 성능을 발휘
- **GBM 대비 빠른 수행 시간**
병렬 수행 및 다양한 기능으로 GBM에 비해 빠른 수행 성능을 보장
(하지만 다른 머신러닝 알고리즘에 비해 빠르다는 의미는 아님)
- **과적합 규제 (Regularization)**
XGBoost자체에 과적합 규제 기능 → 과적합에 좀 더 강한 내구성
- **Tree pruning (가지치기)**
일반적으로 GBM은 부정 손실 발생 시 분할 중단 → 그래도 많은 분할 생성 가능
max_depth로 분할 깊이 조정 가능 + tree pruning으로 더이상 긍정 이득이 없는 분할을 가지치기해서 분할 수를 더 줄이기 가능
- **자체 내장 교차 검증**
반복 수행 시마다 내부적으로 학습 데이터 세트와 평가 데이터 세트(Validation)에 대한 교차 검증을 수행
→ 최적화된 반복 수행 횟수를 가질 수 있음
(지정된 반복 횟수가 아니라, 교차 검증을 통해 평가 데이터 세트의 평가 값이 최적화되면 반복을 조기 중단 (earlystopping) 가능)
- **결손값 자체 처리**
결손값을 자체 처리할 수 있다.



파이썬 래퍼 XGBoost 하이퍼 파라미터

GBM와 유사한 하이퍼 파라미터 + early stopping, 과적합 규제 하이퍼 파라미터 등이 추가

⇒ 사이킷런 래퍼 XGBoost모듈의 일부 파라미터와 다르므로 주의 필요

DMatrix

파이썬 래퍼 XGBoost만의 전용 데이터 객체

- Numpy / Pandas로 되어 있는 학습용, 검증, 테스트용 데이터 세트를 모두 DMatrix로 생성하여 모델에 입력해 주어야함
- 현재는 넘파이/DataFrame/Series 기반으로도 DMatrix 생성 가능.
- 주요 입력 파라미터
 - data : 피쳐 데이터 세트
 - label : 레이블 데이터 세트 (분류) / 숫자형 데이터 세트 (회귀)

⇒ 넘파이, DataFrame, Series외에도 libsvm txt 포맷 파일, xgboost 이진 버퍼 파일을 파라미터로 입력 받아 변환 할 수 있음

하이퍼 파라미터 정리

• 일반 파라미터

: 실행 시 스레드 개수나 silent 모드 등에 대한 선택을 위한 파라미터, 디폴트 값을 변경하는 경우 거의 없음

주요 파라미터	설명
booster	gbtree (tree base model) 또는 gblinear (linear model) 선택, 디폴트는 gbtree
silent	디폴트는 0, 출력 메시지 필요 없으면 1로 설정
nthread	CPU의 실행 스레드 개수 조정, 디폴트는 전체 사용

• 부스터 파라미터

: 트리 최적화, 부스팅, regularization등과 관련 파라미터들

주요 파라미터	설명
eta [default=0.3, alias: learning_rate]	GBM의 학습률과 같은 파라미터 ■ 0에서 1 사이 값 지정 ■ 파이썬 래퍼 기반 사용시 디폴트는 0.3 ■ 사이킷런 래퍼 클래스의 경우 eta가 아니라 learning_rate 파라미터 사용, 디폴트는 0.1 ■ 보통 0.01 ~ 0.2 사이의 값을 선호
num_boost_rounds	GBM의 n_estimators와 같은 파라미터
min_child_weight [default=1]	트리에서 추가적으로 가지를 나눌지 결정하기 위해 필요한 데이터들의weight 총합 ⇒ 클수록 분할을 자제, 과적합 조절용
gamma [default=0, alias: min_split_loss]	트리의 리프 노드를 추가적으로 나눌지를 결정할 최소 손실 감소 값 ■ 해당 값보다 큰 loss가 감소된 경우 리프 노드를 분리 ■ 값이 클수록 과적합 감소 효과
max_depth [default=6]	트리기반 알고리즘과 같음 ■ 0 지정시 깊이 제한 X ■ 보통 3~10사이의 값 적용
sub_sample [default=1]	GBM의 subsample과 동일 - 데이터 샘플링 비율 지정 ■ 일반적으로 0.5~ 1 사이의 값 사용
colsample_bytree [default=1]	GBM의 max_features와 유사 - 트리 생성에 필요한 칼럼을 임의로 샘플링 ■ 매우 많은 피처가 있는 경우 적용
lambda [default=1, alias: reg_lambda]	L2 Regularization적용 값 ■ 피처가 많을 경우 적용을 검토 ■ 값이 클수록 과적합 감소 효과
alpha [default=0, alias: reg_alpha]	L1 Regularization 적용 값 ■ 피처가 많을 경우 적용을 검토 ■ 값이 클수록 과적합 감소 효과
scale_pos_weight [default=1]	특정 값으로 치우친 비대칭한 클래스로 구성된 데이터 세트의 균형을 유지하기 위한 파라미터

• 학습 태스크 파라미터 : 학습 수행 시 객체 함수, 평가 지표 등을 설정

주요 파라미터	설명
objective	최솟값을 가져야 할 손실 함수를 정의 (이진/다중 분류에 따라 달라짐)
binary:logistic	이진 분류일 때 적용
multi:somftmax	다중 분류일 때 적용 → num_class 클래스 개수 파라미터 지정 필수
multi:softprob	multi:softmax와 유사하지만, 개별 레이블 클래스의 해당되는 예측 확률 반환
eval_metric	검증에 사용되는 함수 정의 ■ 회귀의 경우 rmse ■ 분류의 경우 error

- eval_metric 검증 함수 유형
 - rmse : Root Mean Square Error
 - mae : Mean Absolute Error
 - logloss : Negative log-likelihood
 - error : Binary classification error rate (0.5 threshold)
 - merror : Multiclass calssification error rate
 - mlogloss : Multicalss logloss
 - auc : Area under the curve

➡ 뛰어난 알고리즘일수록 파라미터를 튜닝할 필요가 적음 & 파라미터 튜닝에 들이는 수고 대비 성능 향상 효과가 높지 않은 경우가 대부분

과적합 문제가 심할 경우 적용 고려 사항

- eta 값 낮추기 (eta 낮출 경우, num_round(또는 n_estimators)는 반대로 높여줘야 함)
- max_depth 값 낮추기
- min_child_weight 값 높이기
- gamma 값 높이기
- subsample, colsample_bytree의 조정도 과적합 문제에 도움이 될 수 있음

Early Stopping 기능

XGBoost에는 자체적으로 여러 기능을 가지고 있음

기본 GBM의 경우, n_estimator(또는 num_boost_rounds)에 지정된 횟수만큼 반복적으로 학습 오류를 감소 시키며, 중간에 멈추지 않고 지정된 횟수를 다 완료

⇒ XGBoost / LigthGBM은 조기 중단 기능이 있어서 지정 횟수에 도달하지 않더라도

예측 오류가 개선되지 않으면 중간에 중단해 수행 시간을 개선함

⇒ 예를 들어, 반복 횟수를 200, 조기 중단 파라미터 값을 50으로 설정하면 1~200까지 부스팅을 반복하다가 50회를 반복하는 동안 학습 오류가 감소하지 않으면 종료

그 외 기능

- `plot_importance(xgb_model, ax=ax)`

피처의 중요도를 막대그래프 형식으로 출력

- `xgboost.cv()`

사이킷런의 GridSearchCV와 유사하게 데이터 세트에 대한 교차 검증 수행 후 최적 파라미터 탐색

▼ 파라미터

- params (dict) : 부스터 파라미터
- dtrain (DMatrix) : 학습데이터
- num_boost_round (int) : 부스팅 반복 횟수
- nfold (int) : CV 폴드 개수
- stratified (bool) : CV 수행 시 층화 표본 추출 수행 여부
- metrics (string or list of strings) : CV 수행 시 모니터링할 성능 평가 지표
- early_stopping_rounds (int) : 조기 중단을 활성화시킴, 반복 회수 지정

⇒ DataFrame 반환

📄 사이킷런 래퍼 XGBoost의 개요 및 적용

사이킷런의 기본 Estimator를 그대로 상속해 만들

- Estimator와 동일하게 fit() 과 predict() 만으로 학습과 예측이 가능
- 사이킷런의 다른 유틸리티 그대로 사용 가능

네이티브 하이퍼 파라미터와 차이

- eta → learning_rate
- sub_sample → subsample
- lambda → reg_lambda
- alpha → reg_alpha
- early stopping에서 eval_set에 문자열 안 넣어도 됨

7 LightGBM



LightGBM?

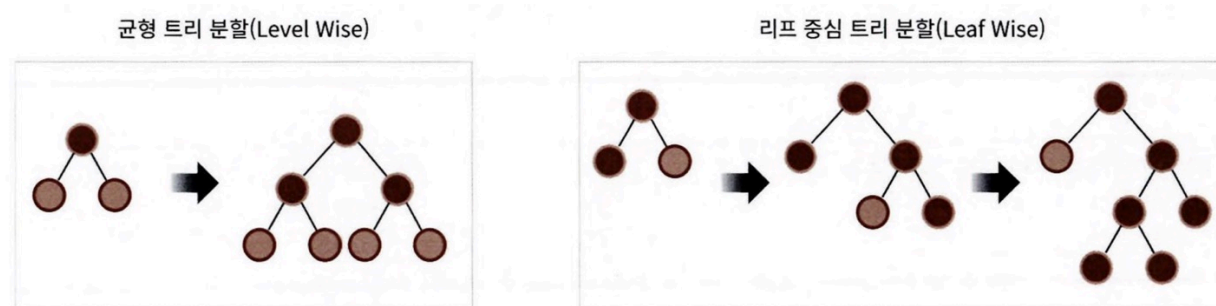
XG 부스트와 함께 부스팅 계열 알고리즘에서 가장 각광을 받고 있음

```
import lightgbm
from lightgbm import LGBMClassifier
```

XGBoost와 마찬가지로 초기에는 파이썬 래퍼용만 개발, 이후 사이킷런 래퍼를 추가로 개발
→ lightgbm패키지 내 두개의 모듈이 모두 있음

XGBosst 대비 장점

- 더 빠른 학습과 예측 시간
- 저 작은 메모리 사용량
- 카테고리형 피처의 자동 변환과 최적 분할 (원-핫 인코딩 등을 사용하지 않고도 카테고리형 피처를 최적으로 변환하고 이에 따른 노드 분할 수행)
- 리프 중심 트리 분할 방식을 사용



- 균형 트리 분할 : 최대한 균형 잡힌 트리를 유지하면서 분할
→ 깊이 최소화, 오버피팅에 강한 구조

- 리프 중심 트리 분할 : 균형을 맞추지 않고 최대 손실 값(max delta loss)을 가지는 리프 노드를 지속적으로 분할
→ 예측 오류 손실을 최소화

LightGBM 하이퍼 파라미터

XGBoost와 매우 유사 / 리프노드가 계속 분할되면서 트리의 깊이가 깊어지므로, 이에 따른 하이퍼 파라미터 설정이 더 필요

주요 파라미터	설명
<code>num_iterations</code> [default = 100]	반복 수행하려는 트리의 개수를 지정 크게 지정할수록 예측성능 증가, 너무 크면 과적합 가능성 (사이킷런 호환 클래스에서는 <code>n_estimators</code>)
<code>learning_rate</code> [default = 0.1]	0에서 1사이의 값을 가지는 학습률 값
<code>max_depth</code> [default = -1]	트리 깊이 설정 Leafwise기반이므로 상대적으로 더 깊게 들어감
<code>min_data_in_leaf</code> [default = 20]	리프노드가 되기 위해 필요한 최소 레코드 수, 과적합 제어 (사이킷런 호환 클래스 LigthGBMClassifier에서는 <code>min_child_samples</code>)
<code>num_leaves</code> [default = 31]	하나의 트리가 가질 수 있는 최대 리프 개수
<code>boosting</code> [default = gbdt]	부스팅의 트리를 생성하는 알고리즘을 기술 ■ gbdt : 일반적인 그래디언트 부스팅 결정 트리 ■ rf : 랜덤 포레스트
<code>bagging_fraction</code> [default = 1.0]	데이터 샘플링 비율 지정 (LightGBMClassifier에서는 <code>subsample</code>)
<code>feature_fraction</code> [default = 1.0]	개별 트리 학습마다 무작위로 선택하는 피쳐 비율 (Light GBMClassifier에서는 <code>colsample_bytree</code>)
<code>lambda_l2</code> [default = 0.0]	L2 regulation 제어를 위한 값 → 피쳐 개수가 많을 경우 적용. 값이 클수록 과적합 감소 (LightGBMClassifier에서는 <code>reg_lambda</code>)
<code>lambda_l1</code> [default = 0.0]	L1 regulation 제어를 위한 값 → L2와 마찬가지로 (LightGBMClassifier에서는 <code>reg_alpha</code>)
<code>objective</code>	Learning Task 파라미터 → 최솟값을 가져야할 손실 함수를 정의, 회귀, 다중 분류, 이진 분류인지에 따라서 지정됨

하이퍼 파라미터 튜닝 방안

- num_leaves의 개수를 중심으로 min_child_samples(min_data_in_leaf), max_depth를 함께 조정하면서 모델의 복잡도를 줄이는 것이 기본 튜닝 방안
 - 일반적으로 num_leaves의 개수를 높이면 정확도가 높아지지만, 트리가 깊어지고 복잡도가 커져 과적합 영향도가 증가
 - min_data_in_leaf (min_child_sampels)는 과적합을 제어하기위한 주요 파라미터.
num_leaves와 학습 데이터 크기에 따라 달라지지만, 보통 큰 값으로 설정하면 깊어지는 것을 방지
 - max_depth는 명시적으로 깊이의 크기를 제한, 위 두 파라미터와 결합해 과적합 개선에 사용
- learning_rate를 작게 + n_estimators를 크게하는 것이 부스팅 계열 튜닝에서 가장 기본적 방법
- 이 외에도 reg_lambda, reg_alpha와 같은 규제를 적용하거나, colsample_bytree, subsample 파라미터 적용도 가능

파이썬 래퍼 LightGBM과 사이킷런 XGBoost, LightGBM 파라미터 비교

LightGBM은 사이킷런과의 호환을 위해 LGBMClassifier와 LGBMRegressor 클래스를 래퍼 클래스로 생성함

→ LigthGBM은 XGBoost와 유사한 기능이 있어 XGBoost 클래스에 맞추어 변경

→ 많은 파라미터가 같다

유형	파이썬 래퍼 LightGBM	사이킷런 래퍼 LightGBM	사이킷런 래퍼 XGBoost
파라미터명	num_iterations	n_estimators	n_estimators
	learning_rate	learning_rate	learning_rate
	max_depth	max_depth	max_depth
	min_data_in_leaf	min_child_samples	N/A
	bagging_fraction	subsample	subsample
	feature_fraction	colsample_bytree	colsample_bytree
	lambda_l2	reg_lambda	reg_lambda
	lambda_l1	reg_alpha	reg_alpha
	early_stopping_round	early_stopping_rounds	early_stopping_rounds
	num_leaves	num_leaves	N/A
	min_sum_hessian_in_leaf	min_child_weight	min_child_weight

사이킷런 래퍼 클래스가 없던 초기 파이썬 래퍼 LightGBM과 사이킷런 LightGBM 하이퍼파라미터 정리

8 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝



Grid Search 방식의 단점

튜닝해야 하이퍼 파라미터 개수가 많을 경우 최적화 수행 시간이 오래 걸림
+ 개별 파라미터의 값의 범위나 학습 데이터가 대용량일 경우 더욱 늘어남

- ➡ XGBoost나 LightGBM은 하이퍼 파라미터 개수가 상대적으로 많은 알고리즘
- ➡ GridSearch 방식으로는 매우 많은 시간이 소모됨, 따라서 다른 방식을 적용



베이지안 최적화 개요

목적 함수 식을 제대로 알 수 없는 블랙 박스 형태의 함수

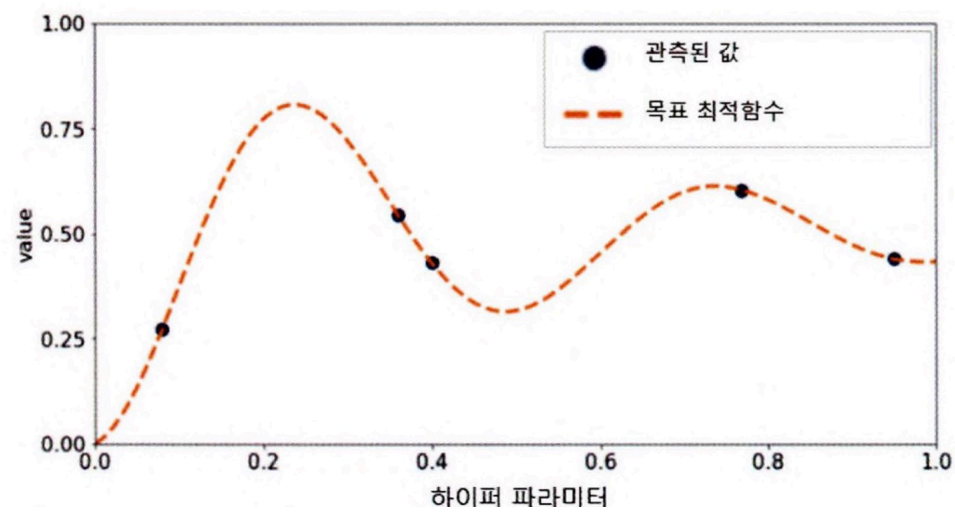
→ 최대/최소 반환값을 만드는 최적의 입력값을 효율적으로 찾아주는 방식

- 베이지안 확률에 기반
 - 새로운 데이터를 입력받았을 때 최적 함수를 예측하는 사후 모델을 개선해 나가며 최적 함수 모델을 만듦
- 베이지안 최적화 구성 요소
 - 대체 모델 (Surrogate Model)
 - : 획득 함수로부터 최적 함수를 예측할 수 있는 입력값을 추천 받은 뒤, 최적 함수 모델을 개선
 - 일반적으로 가우시안 프로세스 (Gaussian Process) 적용 / HyperOpt는 트리 파르젠 사용
 - 획득 함수 (Acquisition Function)
 - : 개선된 대체 모델을 기반으로 최적 입력값을 계산
- 이때 말하는 '입력값' = 하이퍼 파라미터

동작 단계

- **Step1**

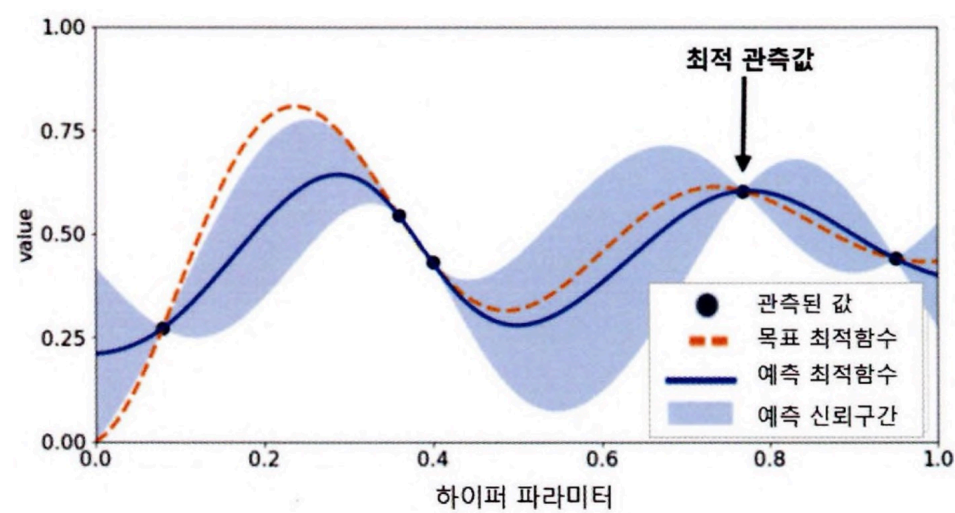
최초에는 랜덤하게 하이퍼 파라미터들을 샘플링하고 성능 결과를 관측



➡ 검은색 원은 특정 하이퍼 파라미터가 입력되었을 때 관측된 성능 지표 결과값

• **Step2**

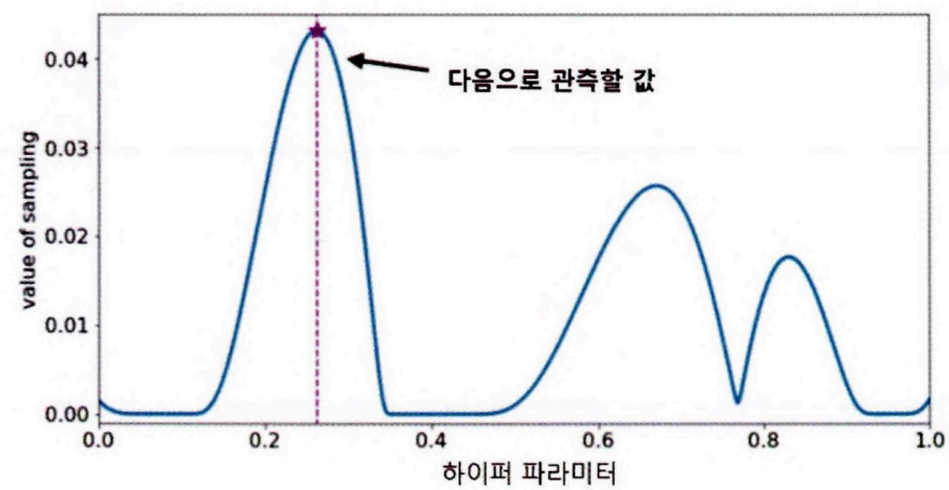
관측된 값을 기반으로 대체 모델은 최적 함수를 추정



➡ 최적 관측값은 y축 value에서 가장 높은 값을 가질 때의 하이퍼 파라미터

• **Step3**

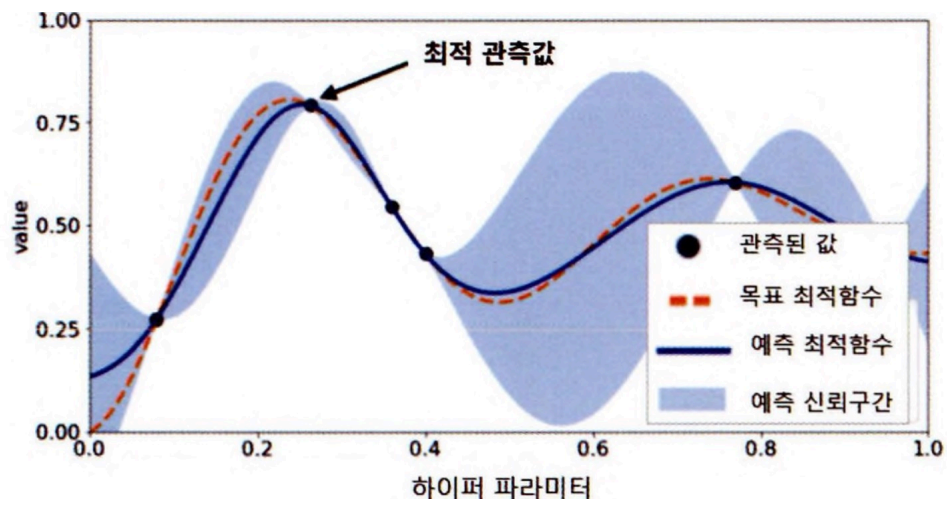
추정된 최적 함수를 기반으로 획득함수(Acquisition Function)가 다음으로 관측할 하이퍼 파라미터 값을 계산



➡ 더 큰 값을 가질 가능성이 높은 지점을 찾아서 대체 모델에 전달

• **Step4**

획득 함수로부터 전달된 하이퍼 파라미터를 수행하여 관측된 값을 기반으로 대체 모델은 갱신 됨 → 다시 최적 함수를 예측 추정



- 3, 4번째 단계를 특정 횟수만큼 반복
→ 불확실성 개선 / 최적 함수 정확히 추정 가능

HyperOpt 사용하기

베이지안 최적화를 적용할 수 있는 대표적 파이썬 패키지
→ HyperOpt / Bayesia Optimization / Optuna 등

HyperOpt 주요 활용 로직

```
from hyperopt import hp
```

- 첫째는 입력 변수명과 입력값의 **검색 공간(Search Space)** 설정

검색 공간 제공 대표 함수들	설명
<code>hp.quniform(label, low, high, q)</code>	label로 지정된 입력값 변수 검색 공간을 low에서 high까지 q의 간격을 가지고 설정
<code>hp.uniform(label, low, high)</code>	최솟값 low에서 high까지 정규 분포 형태의 검색 공간 설정
<code>hp.randint(label, upper)</code>	0~upper까지 random한 정숫값으로 검색 공간 설정
<code>hp.loguniform(label, low, high)</code>	$\exp(\text{uniform}(\text{low}, \text{high}))$ 값을 반환, 반환값의 log 변환 값은 정규 분포 형태를 가지는 검색 공간 설정
<code>hp.choice(label, options)</code>	검색 값이 문자열 또는 문자열과 숫자값이 섞여 있을 경우 설정 options는 리스트나, 튜플형태

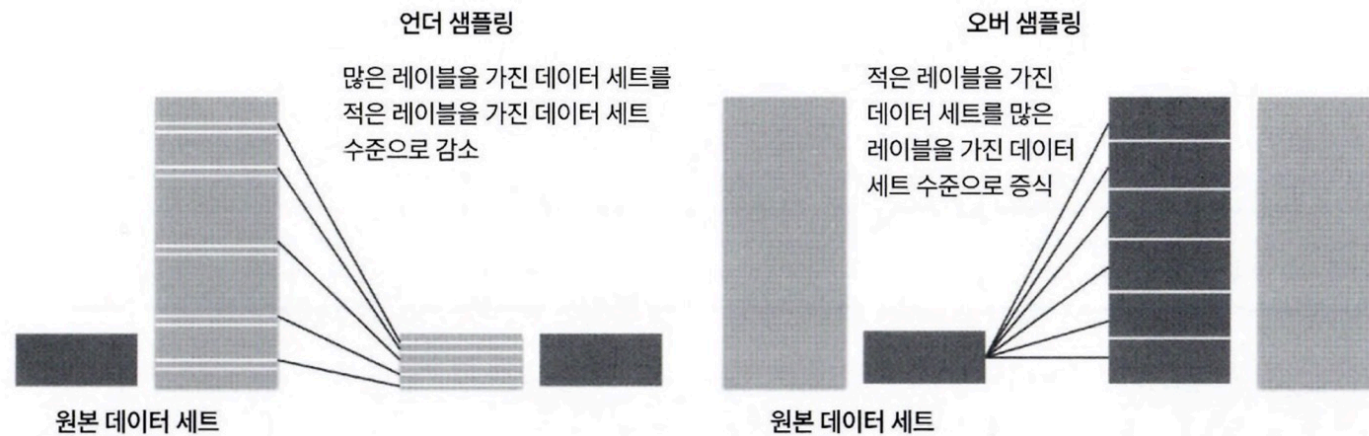
- 둘째는 **목적 함수 (Objective Function)**의 설정
변숫값과 검색공간을 가지는 딕셔너리를 인자로 -> 특정 값을 반환하는 구조
- 마지막으로 목적 함수의 반환 **최솟값**을 가지는 **최적 입력값을 유추**하는 것
→ 베이지안 최적화 기법에 기반

<code>fmin()</code> 주요 인자	설명
fn	정의된 목적 함수
space	정의된 검색 공간 딕셔너리
algo	베이지안 최적화 적용 알고리즘 기본적으로 tpe.suggest (=Tree of Parzen Estimator) 최적화 알고리즘
max_evals	최적 입력값을 찾기 위한 입력값 시도 횟수
trials	최적 입력값을 찾기 위해 시도한 입력값 및 해당 입력값의 목적 함수 반환값 결과를 저장하는데 사용 → trials 클래스를 객체로 생성한 변수명 입력
rstase	fmin()을 수행할 때마다 동일한 결과값을 가질 수 있도록 설정하는 랜덤 시드 값

10 분류 실습 - 언더 샘플링과 오버 샘플링의 이해

레이블이 불균형한 경우

- 제대로 다양한 유형을 학습하지 못하고 특정 레이블로 치어친 학습을 수행하게 돼, 제대로 된 이상 데이터 검출이 어려워짐
- 이때 적절한 학습 데이터를 확보하기 위해 사용하는 방안이 오버 샘플링과 언더 샘플링
- 오버 샘플링 방식이 더 많이 사용됨



• 언더 샘플링

: 많은 데이터 세트를 적은 데이터 세트 수준으로 감소

⇒ 너무 많은 정상 레이블 데이터를 감소시켜서 정상 레이블의 경우 제대로 된 학습을 수행할 수 없는 문제가 발생 가능

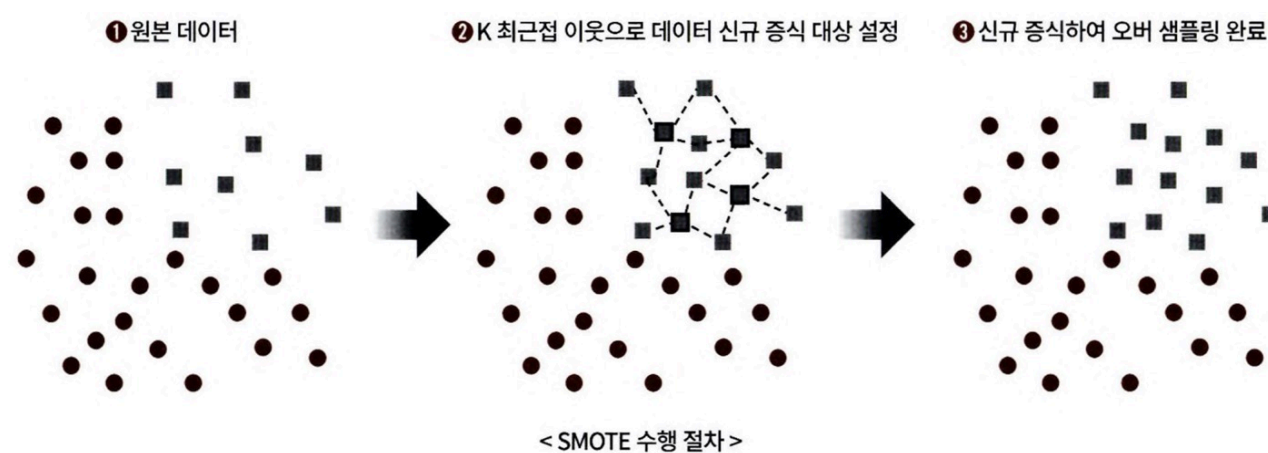
• 오버 샘플링

: 적은 데이터 세트를 증식하여 학습을 위한 충분한 데이터를 확보하는 방법

⇒ 원본 데이터들의 피처값들을 약간 변경하여 증식

◦ SMOTE (Synthetic Minority Over-sampling Technique)

: 적은 데이터 세트에 있는 개별 데이터들의 K Nearest Neighbor를 찾아서 이 데이터와 K 개 이웃들의 차이를 일정 값으로 만들어 기존 데이터와 약간 다른 새로운 데이터들을 생성



`imbalanced-learn` 패키지에서 구현

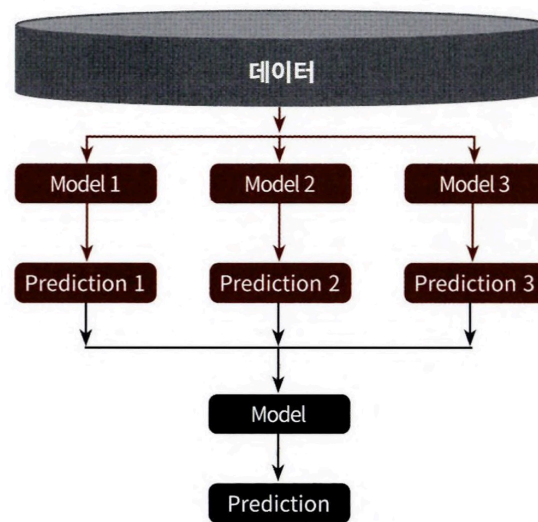
1 1 스택킹 앙상블



스태킹 앙상블 vs 배깅/부스팅

- 공통점
개별적인 여러 알고리즘을 결합해 예측 결과를 도출
- 차이점
개별 알고리즘으로 예측한 데이터를 기반으로 다시 예측을 수행
⇒ 개별 알고리즘의 결과 데이터 세트를 최종적 메타 데이터 세트로 생성 ⇒ 별도의 ML알고리즘으로 최종 학습을 수행
= 메타 모델

스태킹 모델 구성



• 개별적인 기반 모델

: 많은 개별 모델이 필요

• 최종 메타 모델

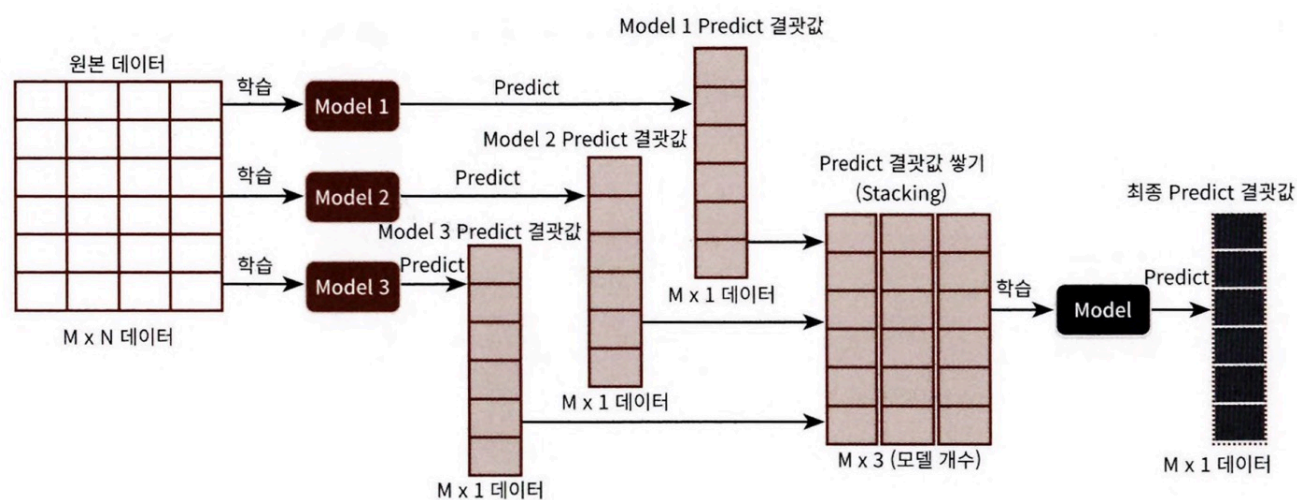
: 개별 기반 모델의 예측 데이터를 학습 데이터로 만들어 학습하는 최종 메타 모델

⇒ 여러 개별 모델의 예측 데이터를 각각 스태킹 형태로 결합해 최종 메타 모델의 학습용 피쳐 데이터 세트와 테스트용 데이터 세트를 만드는 것이 핵심

⇒ 조금이라도 성능 수치를 높여야 할 경우 사용됨

예측 과정

M개의 로우, N개의 피쳐(칼럼)를 가진 데이터 세트에 스태킹 앙상블 적용 시



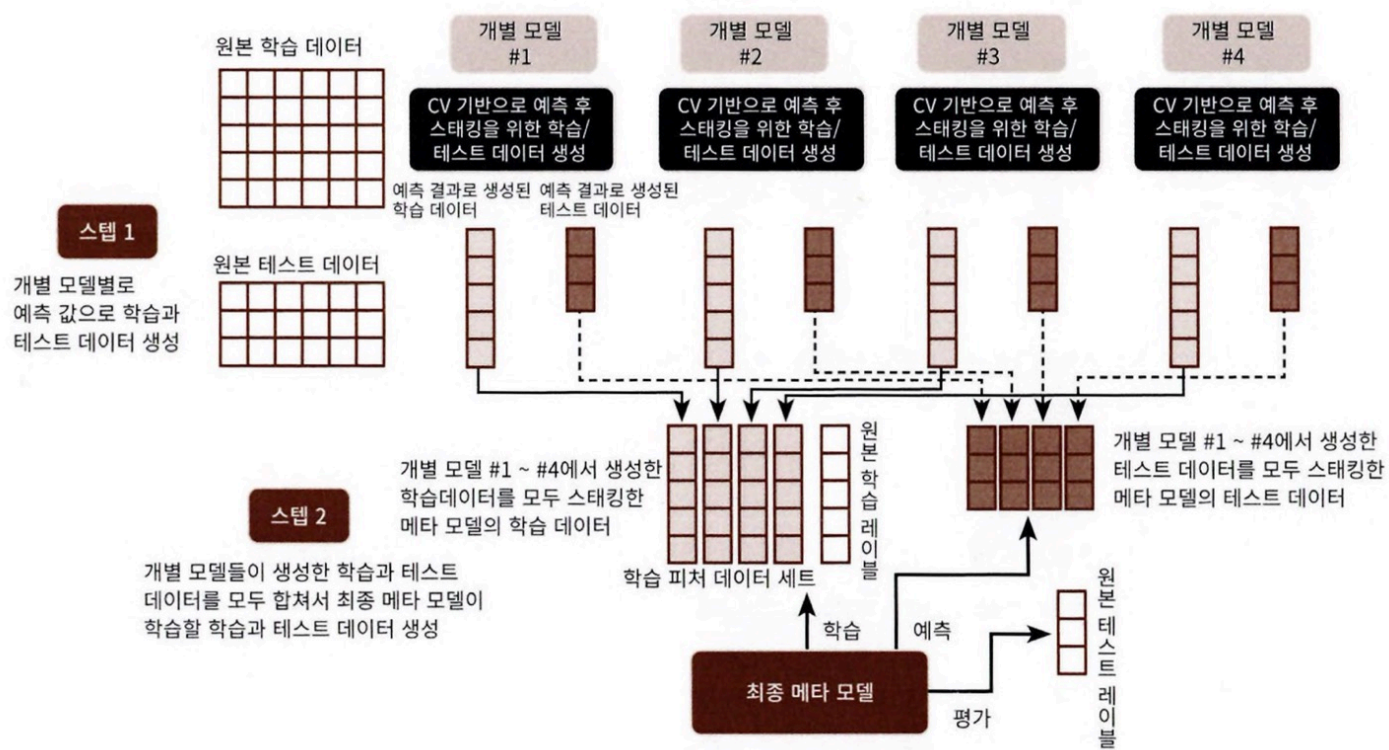
- 3개의 각 모델 별로 각각 학습을 시킨 뒤, 예측을 수행하면 각각 M개의 로우를 가진 1개의 레이블 값을 도출
- 모델 별로 도출된 예측 레이블 값을 다시 합해서 (스태킹) 새로운 데이터 세트를 생성

- 이렇게 스택된 데이터 세트에 대해 최종 모델을 적용해 최종 예측을 수행

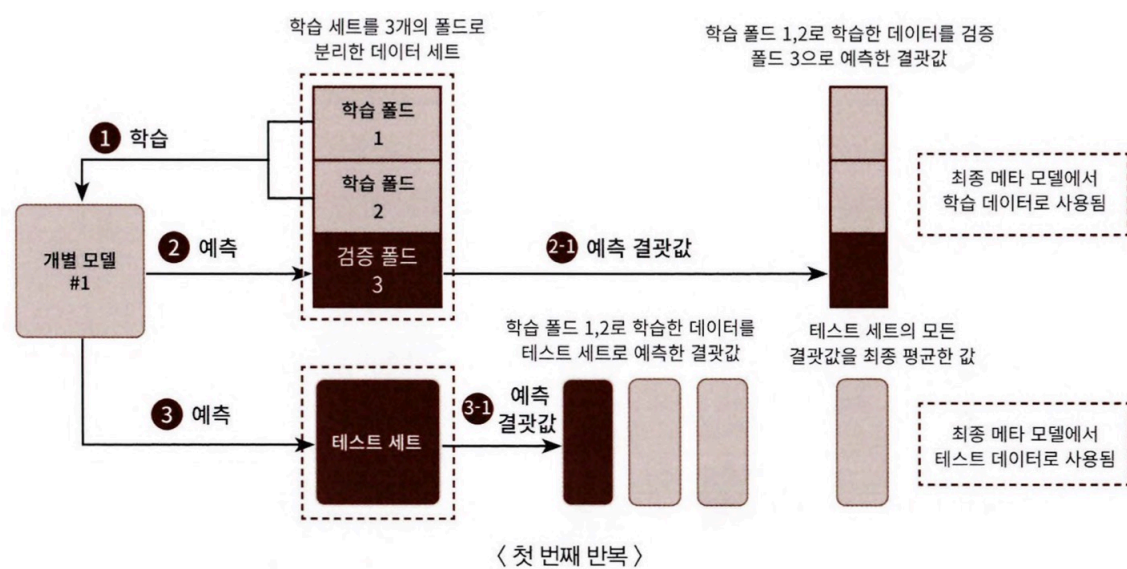
CV 세트 기반의 스택킹

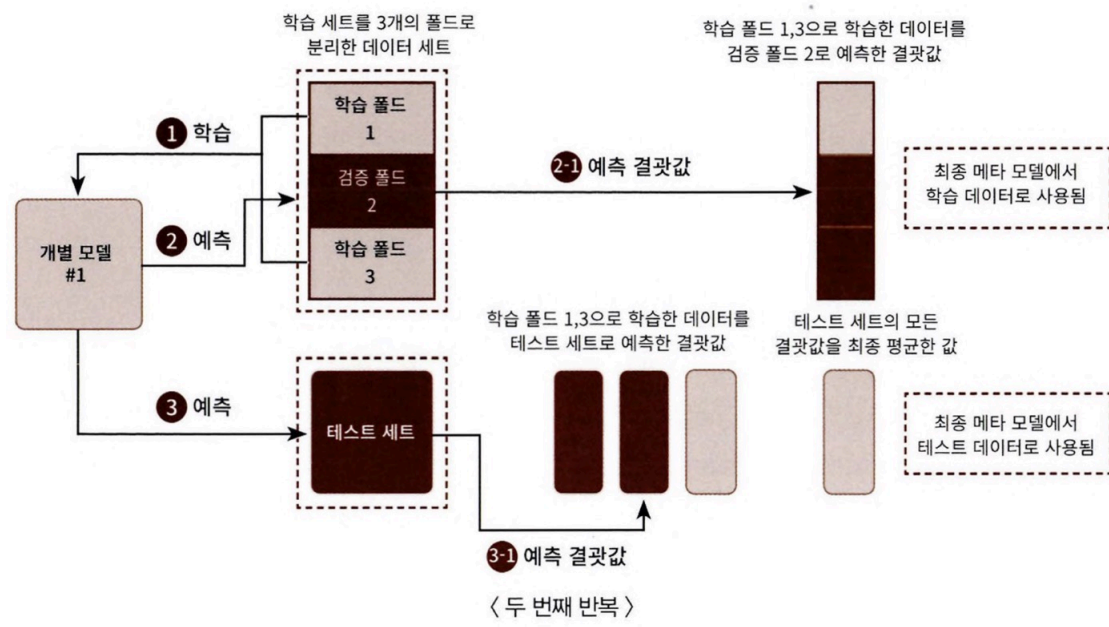
- 과적합을 개선하기 위해 최종 메타 모델을 위한 데이터 세트를 만들때, 교차검증 기반으로 예측된 데이터 세트를 이용
- 개별 모델들이 각각 교차 검증으로 메타 모델을 위한 학습용 스택킹 데이터 생성과 예측을 위한 테스트용 스택킹 데이터를 생성한 뒤, 이를 기반으로 메타 모델이 학습과 예측을 수행
- 분류 뿐만 아니라 회귀에도 적용 가능

과정

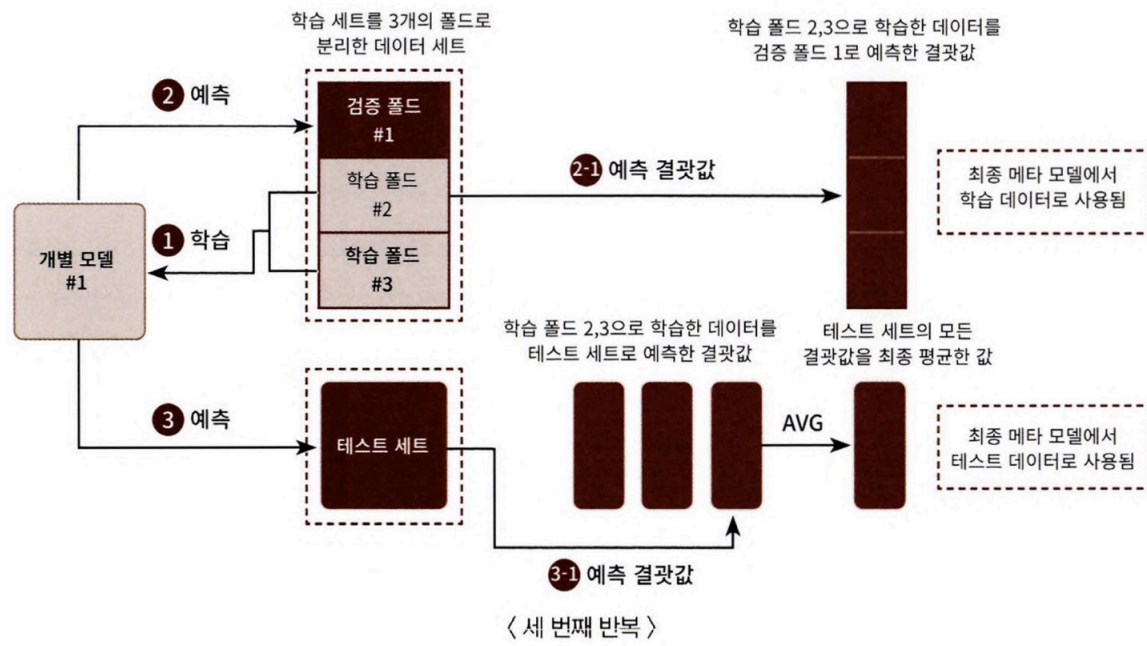


1. 각 모델 별로 원본 학습/테스트 데이터를 예측한 결과 값을 기반으로 메타 모델을 위한 학습용/테스트용 데이터를 생성
 - a. 학습용 데이터를 3개의 폴드로 나누되, 2개의 폴드는 학습용, 나머지 1개는 val용을 위해 사용
 - b. 학습된 개별 모델은 검증 폴드 1개 데이터를 예측, 결과 저장 ⇒ 3번의 폴드 반복
⇒ 메타 모델을 위한 학습 데이터로 사용됨





c. 3번 반복하면서 예측값의 평균으로 최종 결과값을 생성하고 이를 메타 모델을 위한 테스트 데이터로 사용



2. 개별 모델들이 생성한 학습용 데이터를 모두 스택킹 형태로 메타 모델이 학습할 최종 학습용 데이터 세트를 생성
3. 각 모델들이 생성한 테스트용 데이터를 모두 스택킹 형태로 합쳐서 메타 모델이 예측할 최종 데이터 세트를 생성
4. 메타 모델이 최종적으로 생성된 학습 데이터 세트와 원본 학습 데이터의 레이블 데이터를 기반으로 학습
5. 최종적으로 생성된 테스트 데이터 세트를 예측하고 원본 테스트 데이터의 레이블 데이터를 기반으로 평가